Algorithms for In-Place, Consistent Network Update

Kedar S. Namjoshi
Nokia Bell Labs
Murray Hill, NJ, USA
kedar.namjoshi@nokia-bell-labs.com

Sougol Gheissi
Johns Hopkins University
Baltimore, MD, USA
sgheiss1@jhu.edu

Krishan Sabnani
Johns Hopkins University
Baltimore, MD, USA
ksabnan2@jhu.edu

ABSTRACT

Network configurations are regularly updated in response to issues such as congestion, failures, network changes, and modifications to security policies. We present a simple distributed algorithm for network update that operates *on the fly* and *in place*, and guarantees *strong route-consistency*. Existing methods are either weakly consistent, or do not operate in place and require excessive memory.

We prove that our method, called a *causal update*, appears to take effect instantaneously at a quiescent network state, even though it is actually carried out over time and interleaved with normal network operation. This property ensures per-packet consistency: i.e., a packet is processed either entirely within the old configuration or within the new one, never by a mix of the two. The price paid for these strong guarantees is that the algorithm may be forced to occasionally drop packets for consistency. We prove that forced drops cannot be avoided: any in-place and on-the-fly update method must drop packets or violate consistency. We show how to exploit network structure and update characteristics to reduce or entirely eliminate packet drops. Simulation experiments indicate that the (temporary) loss in throughput from forced packet drops falls within acceptable limits.

CCS CONCEPTS

• **Networks** → **Network protocol design**; **Network management**; • **Computing methodologies** → *Distributed algorithms.*

KEYWORDS

Network Updates, Route Consistency, Formal Analysis

ACM Reference Format:
Kedar S. Namjoshi, Sougol Gheissi, and Krishan Sabnani. 2024. Algorithms for In-Place, Consistent Network Update. In *ACM SIGCOMM 2024 Conference (ACM SIGCOMM '24), August 4–8, 2024, Sydney, NSW, Australia.* ACM, New York, NY, USA, 14 pages. https://doi.org/10.1145/3651890.3672266

1 INTRODUCTION

A network routes packets through nodes with different functionalities, such as switches, routers, and firewalls. Occasionally, the routing rules must be modified. Typically, that is done to re-route traffic in response to congestion, failures, or changes to security policies. It is well understood that if nodes are updated in a haphazard order, the network may misdirect traffic, violate security policies, create routing loops, or encounter other failures. While the failures are temporary, they are nonetheless concerning, as they may give rise to security holes (e.g., from misdirected traffic) or performance penalties (*e.g.,* from routing loops). As a consequence, there is extensive research on determining the right ordering for network updates.

What should "right ordering" mean? The validity of a network update may be defined by the routing properties that are satisfied during the update process. The pioneering work in [31] introduced the criterion of *per-packet consistency*. It requires that every data packet is processed entirely within the old configuration or entirely within the new one. As a consequence, every property determined by packet history is satisfied during an update if it is satisfied in both configurations. This is a powerful result, as the update failures listed above can all be expressed as violations of packet-history properties. For instance, a routing loop is witnessed by a packet history on which a network node occurs twice.

That paper also proposes a two-phase update algorithm. While the algorithm ensures per-packet consistency, it requires every node to support both configurations until the update is complete. That is a serious obstacle in practice, as routers (in particular, high-speed routers) have specialized, expensive hardware with limited memory capacity. (Router memory limitations are a major motivation for new architectures and management approaches [17, 21].) We say that the two-phase algorithm does not operate *in place*, as the configurations co-exist during the update.

A strawman in-place update algorithm would block all traffic into the network, wait for existing packets to drain out of the network, update the configuration, then unblock incoming traffic. This algorithm is in-place and strongly consistent, but it is clearly unworkable in practice. That is because it is not *on the fly*, by which we mean that updates are not interleaved with normal network processing.

Update algorithms that operate in place and on the fly (cf. [7, 13, 15, 18, 26–28, 34, 37]) either support weaker consistency guarantees, such as satisfying only loop-freedom, or provide strong consistency only for some updates, falling back to the two-phase algorithm.

This discussion leads to the fundamental question that is addressed in this paper: *Can a network update algorithm operate in place, on the fly, and guarantee strong consistency?*

We present such an algorithm, called a *causal update.*[1] Indeed, we show the stronger property that in a causal update, the network *appears to update instantaneously* at a globally quiescent state (*i.e.,* one where all internal packet queues are empty), even though in reality nodes are updated over time and the update actions are interleaved with normal network operation. Per-packet consistency

[1]This work does not raise any ethical issues.

ACM SIGCOMM '24, August 4–8, 2024, Sydney, NSW, Australia

ACM ISBN 979-8-4007-0614-1/24/08…$15.00
https://doi.org/10.1145/3651890.3672266

is a consequence of this property. The algorithm is resilient to node and link failures short of a permanent network partition.

The price paid for these strong guarantees is that in some scenarios, packets must be dropped. This occurs when packets are trapped between old and new configurations; such packets must be dropped to satisfy per-packet consistency. We prove that packet drops are *unavoidable* for any update algorithm that is on-the-fly, in-place, and strongly consistent. Although packet drops are a concern, higher-level network and application layers are designed to recover from temporary packet losses. We show how network structure and update characteristics can be exploited to reduce or eliminate packet drops while satisfying strong consistency.

The causal update algorithm is simple and can be implemented efficiently in SDN-enabled network nodes. Simulation-based experiments with a prototype implementation show that forced packet drops reduce network throughput by $11 - 20\%$ temporarily during the update. A causal update always completes in time proportional to the network diameter.

This algorithm naturally handles networks with stateful functionality such as stateful firewalls. It may indeed be applied more generally to update components of a distributed service on the fly. A key requirement is that the service should be resilient to temporary and unpredictable message loss. Networks have such resiliency built in by design.

2 CAUSAL UPDATES

In this section, we formulate the causal update algorithm and establish its consistency properties.

2.1 Network Model

A network is modeled as a finite directed graph, as illustrated in Figure 1. A graph node represents a network function, which may be stateful. A directed edge from a node m to a node n represents a channel for transferring packets from m to n. Channels have unbounded capacity–i.e., a packet can always be sent into a channel. Packets may be lost or reordered in transit, but not duplicated. In an actual network, links are typically viewed as bidirectional and have limits on capacity. A bidirectional link between nodes m and n is modeled as a pair of directed edges, one in each direction. For proofs of consistency, it suffices to view a bounded channel as an unbounded channel that may nondeterministically lose packets.

Packets may enter the network only at designated *entry* nodes, and leave the network only at designated *exit* nodes. The entry and exit nodes form the *interface* of the network with its environment. In Figure 1, nodes W and E are the entry nodes, while nodes N and S are the exit nodes. Packets enter or leave through *interface channels*, which are also indicated in the figure. All other channels are called *internal channels*.

A *network configuration* assigns a computational process to each network node. This process determines how incoming packets are handled at that node. We view the process as a state machine. From its current state, a process may either receive a packet from an input channel, or send a packet to an output channel, or perform an internal action, then transition to its next state. This formulation covers typical network functionality: for instance, a process may implement complex routing rules, alter packet headers, encapsulate packets, or even generate fresh packets–and do all this in a stateful manner. The only (mild) restriction – needed for the consistency proof – is that a state transition can depend only on the current process state and if required, the received packet; but not, for instance, on the content of the channels attached to the node or the number of unprocessed packets in those channels.

A *network state* consists of the internal state of every node process, together with the contents of every channel. A network state is *quiescent* if every internal channel is empty.

2.2 The Network Update Problem

A *network update* consists of (1) a new network configuration along with (2) a per-node transfer function that defines, for every node, how to map the state of its currently assigned process, at the time of update, to a state of the newly assigned process for that node. To make it easier to visualize the update actions, we assign colors to the old and new configurations. The original configuration of the network is called a "Red" configuration and the new one a "Green" configuration. A node updates its configuration by "turning Green" – i.e., by replacing its Red process with a Green process and by initializing the Green process state according to the supplied transfer function.

The *correctness of a network update* is defined indirectly. A (hypothetical) instantaneous update is one that takes effect at a quiescent network state and instantaneously turns every node Green. We take as an axiom that any instantaneous update is correct. An update algorithm is correct if for every network computation containing a single update the observed network behavior (i.e., the sequencing and content of packet transmissions) is indistinguishable from the behavior observed if the same update was carried out instantaneously.

Consider the strawman update method from the Introduction, which is akin to the shutdown-and-restart approach we commonly use for updating personal computers. The method is correct as it may be viewed as occurring instantaneously at the quiescent network state after all existing packets have drained from the network.

In-place update methods must update nodes while the network continues to function. This can lead to performance and security failures if nodes are not updated in the right order. In essence, the problem is caused by asynchrony and concurrency: failures arise from bad interleavings between node updates and packet transmissions.

As a simple example, consider a network with distinct nodes A, B, C, D, X and a security policy that requires all packet transmissions from A to B to avoid node X. Say that the Red configuration has paths ACB and DXB while the Green configuration has path ADB. Both configurations satisfy the security policy. Now suppose that node A is updated first. Until the routes from node D are updated, packets will be routed on the newly active path $ADXB$, violating the security policy. The update also violates per-packet consistency, as this new path belongs neither to the old configuration nor to the new one.

While the strawman algorithm operates in place but not on the fly, the two-phase update algorithms from [19, 31] operate on the fly but not in place. The algorithm requires every node to simultaneously support the Red and the Green configurations. Fresh

packets are tagged as Green when they enter the network, and are processed only by the Green processes. Existing Red packets are processed only by the Red processes. This continues until (assuming no routing loops and no internal packet creation) all Red packets eventually exit the network. At this stage, the Red configuration is removed, leaving the network in the Green configuration. This algorithm is on-the-fly, per-packet consistent, and does not drop packets. The significant drawback is that every node must have sufficient additional memory to simultaneously support both configurations during the update. That is often infeasible in practice: for instance, high-speed routers require specialized, expensive memory, which limits memory capacity. As discussed in depth in Section 5, other known update algorithms either provide weaker guarantees, or generate in-place schedules only for some but not all networks, or operate in place only partially, falling back to the two-phase solution.

This discussion leads to the fundamental question tackled in our work: Is there an update algorithm for a (stateful) network that is per-packet consistent, and operates on the fly and in place? The causal update algorithm described next meets all those requirements.

2.3 The Causal Update Algorithm

2.3.1 Markers. The causal update algorithm relies on special control packets called *markers*. A marker packet reception is the signal for a node to update from the Red to the Green configuration. Intuitively, markers moving through a network form a wavefront, with nodes ahead of the wave still in their Red configuration, while nodes behind the wave have switched to their Green configuration. In practice, a marker may be distinguished from other packets by setting an otherwise unused header bit, or through encapsulation. In this section, we refer to marker packets simply as "markers." All non-marker packets are referred to as "packets."

We make two assumptions on how markers are treated by the network: (M1) Every marker is transmitted reliably through a (possibly lossy) channel, and (M2) On every channel, a marker acts as a fence against reordering. I.e., packets in the channel cannot be reordered across a marker, in either direction. Property (M1) can be enforced by an acknowledgment mechanism for markers, while property (M2) can be enforced by waiting for acknowledgments for all prior messages before sending a marker, and waiting for a marker acknowledgment before sending any new messages.

2.3.2 Update Algorithm. It is helpful to view the algorithm as consisting of a "update shell" surrounding the configured network process at each node. The update shell receives and sends markers, using those events to control which packets are processed by the inner configured process, and when the inner process is updated.

At each node, the update algorithm goes through the following sequence of steps. To simplify the description, we "color" (i.e., assign a state to) the input channels of the node. From the perspective of a node, an input channel is red if no marker has been received from the channel; it turns green once a marker is received on it. Initially, all input channels at a node are colored red.

(1) A Red node waits to receive its first marker. Once the first marker is received by the shell process on one of the input channels, it drops the marker and turns that channel green. We refer to this point as the T_0 point on the timeline for that node.
(2) At some later time T_1, the shell decides to turn the node Green, i.e., it replaces the configured Red process with its updated Green counterpart. The update shell then transmits a marker on every output channel. Only after all markers have been transmitted successfully (at time T_2) does the shell allow the new Green process to processing incoming packets. In a general network, the T_1 point must be independent of whether markers have been received on every input channel; waiting for markers on all channels could lead to a deadlock, as illustrated later.
(3) Between points T_0 and T_1 (i.e., after the node has received its first marker but before it turns Green) the node handles incoming packets as follows.
   (a) Any packet received from a green channel is queued for later processing.
   (b) Any packet from a red channel is processed by the current Red configuration.
   (c) Any marker received on a red channel is dropped and the channel changes color to green.
(4) After point T_2 (i.e., after the node has turned Green), the node handles incoming packets as follows.
   (a) Any packet from a green channel is processed by the new Green configuration. This includes packets that were queued in step 3a.
   (b) Any packet from a red channel is dropped.
   (c) Any marker received on a red channel is dropped and the channel changes color to green.
(5) The update concludes when markers have been received on all input channels (at time T_3). Until the next update, all packets are handled by the Green process.

The algorithm is simple and easy to implement. To start the algorithm, a subset of nodes is supplied with a marker – i.e., every node in that set is externally triggered to point T_0. This initial set of nodes must have the property that every other node in the network graph is reachable from some node in this set. All interface input channels to these initial nodes are marked green. The update is complete once every node and every input channel is green.

Step 4b considers the case of a packet p that arrives on a red channel after the node has turned Green. As a marker has not yet been received on that channel and packets cannot be reordered over markers (property M2), packet p must have been sent when its source node was Red. Informally, packet p is now "trapped" between the Red sender and Green recipient nodes. Processing this packet in the Green configuration would violate per-packet consistency, so it must be dropped.

Delaying the switch from Red to Green for the receiving node (i.e., increasing the gap between points T_0 and T_1) may reduce the number of dropped packets, at the expense of delaying the completion of the network update and increasing the number of packets queued at step 3a, which may lead to buffer overflow drops in practice. (Buffer overflows are analyzed experimentally in Section 4.) Globally, the algorithm is asynchronous and distributed: i.e., updates to different nodes are coordinated only through the

marker transmissions, not according to a common clock or a single controller.

2.3.3 Illustration. We illustrate the algorithm by examining an update computation illustrated in Figures 1(a)-(d). Prior to the update (stage (a)), the network contains red packets p and q on the channels $E \rightarrow N$ and $W \rightarrow S$. The update begins at stage (b) with nodes W and N chosen as the initial nodes, which then issue markers m_0 and m_1 (green squares). In stage (c), the red packet q has passed through the Red configuration at S and exited the network; node S receives marker m_1, turns Green, and issues a marker m_2 on the $S \rightarrow E$ channel; and a new green packet s enters the network through node W. In stage (d), node E turns Green, sends marker m_3 to node N, then processes its incoming packet r. The red packet p is dropped by node N (indicated by a slash) as the incoming channel to node N is still red. The green packet s is processed by the Green node S and exits the network. At the end of this stage, all nodes and channels are green, so the update is complete. Notice that green packets are not processed by Red nodes; nor are red packets processed by Green nodes, so this computation is per-packet consistent.

2.4 Correctness

Termination follows from the property that every node is reachable from an initial node and that markers are transmitted reliably. Hence, every node eventually receives a marker on each of its input channels and turns Green. As markers are propagated in a breadth-first fashion from the initial nodes, the time required for the update to complete is proportional to the diameter of the network graph.

We now proceed to establish the central correctness property of the algorithm. In a nutshell, we will show that events on any computation that contains a completed update may be reordered to obtain another valid computation where the update appears to be instantaneous and occurs at a quiescent network state. The reordering preserves causality between sends and receives of packets, giving the algorithm its name, *causal update*.

2.4.1 Consistent Cuts and Quiescent States. The proof relies on fundamental concepts in distributed systems theory, that of a consistent cut and a quiescent state (cf. [4]).

A local state at a node is defined by the state of the process assigned to the node by the current configuration and the state of the update shell process. A global state of the network is defined by the local states at each node and the contents of each channel. An *execution* of the network is a sequence of global states. Following the standard asynchronous interleaving semantics, adjacent states on an execution are related by a state change at a single node, along with associated changes to the contents of the input and output channels adjacent to that node. An *event* represents either the processing of an incoming packet, the sending of an outgoing packet, or an internal state change. As a network may continue to operate forever, we consider fair infinite executions, fair in the sense that every node has infinitely many transitions on the execution.

The events on an execution must respect some basic causal dependencies. The reception of a message by a node n on a channel from node m to node n must follow the sending of that message at node m on that channel. The events along the execution for any node n occur in a linear order. These relations are commonly represented as a *space-time diagram* induced by the execution. The diagram for the update execution of Figure 1 is depicted in Figure 2. The transitive closure of this basic ordering relation is called the *happens-before* relation; this must be an irreflexive partial order as otherwise an event is causally dependent on itself. Every execution induces such a diagram. However, a diagram represents multiple valid executions: each obtained by linearizing the ordering in a way that respects the happens-before relation.

We refer to a partially ordered space-time diagram as a *computation*. A *thread* of a computation is the linearly ordered sequence of events at a node. The proof of update correctness will be framed in terms of computations and their cuts. A *cut* includes, for each node, all events that occur on the thread for that node from the start of that thread up to a specified point. A cut C is *consistent* if it is causally closed, i.e., if the cause of an event in the cut is also in the cut. Precisely, a cut is consistent if it is downward-closed according to the happens-before relation, i.e., if event e is in C and f happens before e, then f must also be in C.

The importance of a consistent cut is that it is a representation of a reachable global state. Precisely, if C is a consistent cut for a computation σ, there is a linearization of σ which defines an execution on which there is a point t such that all events in C occur before t and all events not in C occur beyond t. The cut C thus defines the global network state at point t. In turn, every global state of a network execution induces a consistent cut in the computation derived from that execution.

A consistent cut is *quiescent* if every internal channel is empty in the global network state defined by the cut.

2.4.2 The Main Theorem.

Theorem 2.1. *Let x be an infinite fair execution with a completed update. There is an infinite fair execution y that is a reordering of events on x on which the update appears to take effect instantaneously at a quiescent state. Executions x and y agree on the ordering of non-update events.*

Proof. Although the causal update algorithm is simple, its proof is lengthy. Hence, we sketch the proof strategy before presenting the full proof.

For *any* update computation δ and number k, let $C_k(\delta)$ be the cut defined by the set of events that occur at or before point T_k on every thread. As the points are ordered by $T_0 < T_1 < T_2 < T_3$ on every thread, it follows that $C_0(\delta) \subseteq C_1(\delta) \subseteq C_2(\delta) \subseteq C_3(\delta)$. Cut C_0 gathers the events that occur up to the first marker reception; cut C_1 gathers the events that occur up to the nodes turning green; cut C_2 gathers all events that occur up to the point where the updated node completes sending markers; and cut C_3 represents the maximum extent of the update, as it is defined by the final marker receptions at each node.

The proof has three parts. By an update event we mean a marker send or receive, the event of turning Green, or a forced packet drop. In Part I, we show that in any update computation, all update events between points T_0 and T_1 on any thread may be *commuted to the right* over non-update events, resulting in a computation where the only events between cuts C_0 and C_1 on the reordered computation are update events. In Part II, we show that update events between points T_2 and T_3 on any thread may be *commuted to the left* over

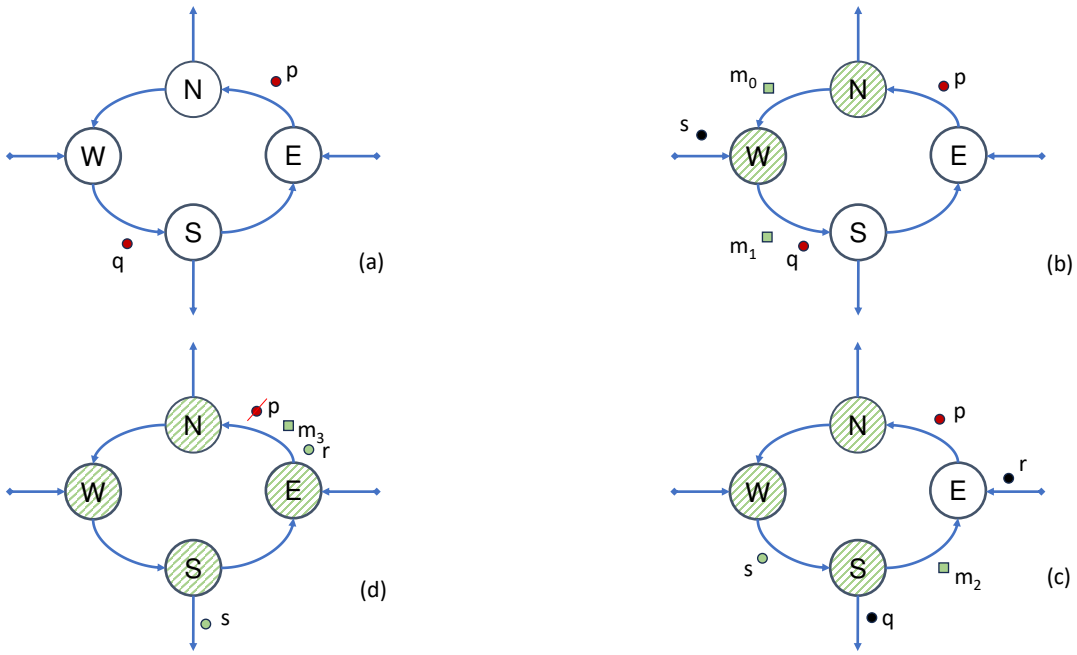

Figure 1: An Update Computation. Updated nodes are shown with (green) cross-hatching. Markers are shown as (green) squares, packets originating from Red nodes as (red, dark) circles, and those originating from Green nodes as (green, light) circles.

non-update events, resulting in a reordered computation where the only events between cuts C_2 and C_3 are update events. By definition, the events between C_1 and C_2 are update events. In Part III, we apply these commutations to derive execution y. We begin with the computation σ_0 derived from execution x. The right-commutations of Part I applied to σ_0 produces a reordered computation σ_1; the left-commutations of Part II applied to σ_1 produce σ_2. Execution y is a linearization of σ_2. We show that it consists of three segments: the initial segment where all nodes are Red; the middle segment consisting only of update events; and the final suffix where all nodes are Green. We show that $C_3(\sigma_2)$ is a quiescent consistent cut, so the middle segment is equivalent to an instantaneous update that occurs at a quiescent network state. The commutations only exchange the relative order of an update and a non-update event; thus the relative order between non-update events is preserved, ensuring per-packet consistency. We now present the detailed argument. (Appendix A shows the reordered form of the computation in Figure 2.)

Part I. The events between points T_0 (inclusive) and T_1 (exclusive) on a thread for a node n are marker receptions that turn channels green, packets received from red channels, and packets sent on output channels. We show that the marker receptions may be *commuted to the right* over the other events.

Consider adjacent events e, f between points T_0 and T_1, where the prior event e is a marker received on channel c and the following event f is a packet that is either sent or received on a channel d. By Step 3c, event e turns channel c green, and, by 3a, no packets from a green channel are processed in the interval between T_0 and T_1. Thus, channels c and d must be distinct. Consider exchanging the order of e and f. Receiving or sending a packet on channel d does not disable receiving a marker on c. The two events affect distinct sub-states at node n: event e affects only the state of the update shell process, and event f only the state of the inner (Red) configuration process. Thus, it is possible to commute the events, leading to the same local state. (The contents of channels c, d differ in the local state between $e; f$ and $f; e$ but, by assumption, the process transition for f is oblivious to this difference.)

We must also show that this reordering does not create a happens-before cycle in the reordered computation. Towards a contradiction, consider a cycle of dependencies in the reordered computation. This cycle must include either e or f, otherwise it exists in the original computation. If it includes only f, it is straightforward to show that a cycle must exist in the original computation. The interesting case is when the cycle includes e. Consider a cycle of the form $q; \alpha; e; \beta; q$. As e is a receive event, the first event b of β must be the following event on the same thread. There are two cases for α. The first case is that the last event of α is f. Then the cycle has the shape $q; \alpha'; f; e; b; \beta'; q$. If α' ends with an event on the thread for n, the cycle $q; \alpha'; e; f; b; \beta'; q$ is in the original computation. Otherwise, the cycle $q; \alpha'; f; b; \beta'; q$ is in the original computation. The other case is that the last event of α is the marker send, which is on a different thread. Then the cycle has the shape $q; \alpha'; a; e; b; \beta'; q$. But then $q; \alpha'; a; e; f; b; \beta'; q$ is a cycle in the original computation.

Part II. We assume that the starting point is a computation that has been transformed according to Part I. The events on this computation between points T_2 (exclusive) and T_3 (inclusive) on a thread

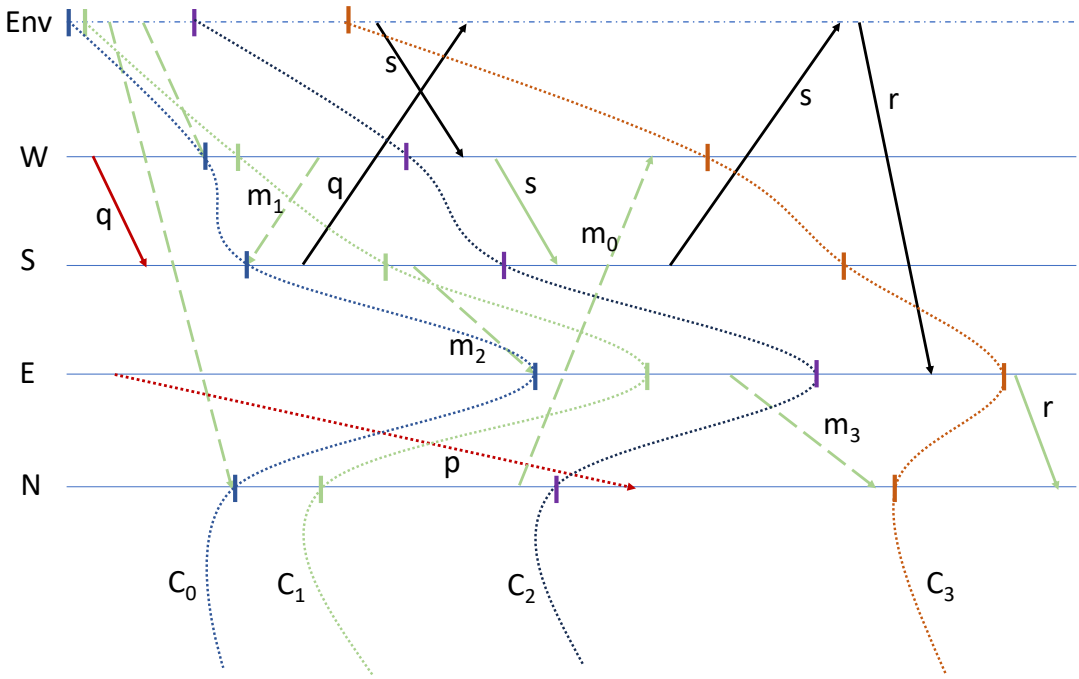

Figure 2: The space-time diagram for the update shown in Figure 1. Marker transmissions are dashed; packet transmissions are solid arrows. Dropped packet p is shown as a dotted arrow. 'Env' represents the network environment. The cuts C_0, C_1, C_2, and C_3 introduced for the proof are illustrated for this computation.

for a node n are marker receptions that turn channels green, packets from red channels that are dropped, packets received on green channels or sent on output channels. We show that the marker receptions and packet drops may be *commuted to the left* over the other events.

Consider adjacent events e, f between points T_2 and T_3 on the thread for some node n, where the prior event e is a packet sent or received on a green channel d, and the following event f is a marker received or a packet dropped from a red channel c. Thus, c and d must be distinct channels, so that performing event f before e does not disable event e, and the resulting state is the same. (The contents of channels c, d do differ in the intermediate state between $e; f$ and $f; e$ but, by assumption, the process transition for event f is oblivious to this difference.)

We must show that reordering does not introduce a cycle. It is straightforward to show that a cycle that does not contain either e or f, or a cycle that contains only e, induces a cycle in the old computation. The interesting case is that of a cycle including f. As f is a receive event, the successor on that cycle must be e. Consider such a cycle $q; \alpha; a; f; e; \beta; q$. There are two cases to consider. If a is the event prior to f on the same thread, then $q; \alpha; a; e; \beta; q$ is a cycle in the old computation.

Now suppose that a is the send event corresponding to f. If f is a marker reception, its corresponding send must lie within C_2 as all marker sends are within C_2. If f is a forced drop, then as the marker on that channel acts as a fence, the origin of f must also be within C_2. Thus, in either case, event a lies in C_2. As f is outside C_2, it follows that on the cycle, there must be a send event g outside C_2 with its corresponding receive event inside C_2. But the marker for that channel is sent inside C_2 and thus precedes the event g on its timeline, so this transmission must be that of a packet. By the rearrangement of Part I, the interval between T_0 and T_2 consists only of update events, so this packet must be received prior to T_0, while the marker is received at or after T_0. Hence, the packet is reordered with respect to the marker, which contradicts assumption (M2).

Part III. After the reorderings of parts I and II, in the resulting computation, all update events lie between cuts C_0 and C_3, and no non-update events are between those two cuts.

We show that C_3 is a consistent cut in this computation. If not, there is a packet send event outside C_3 from a node m with a corresponding receive event at a node n inside C_3. The marker for the channel from m to n is sent inside C_2 and thus precedes the packet on the channel. However, the corresponding marker receive event must occur at or after point T_0 on the thread for n, while the packet receive must occur prior to point T_0 on that thread. Hence, the packet is reordered with respect to the marker, which contradicts assumption (M2).

We also show that C_3 is quiescent. If not, there is a send event at some node m within C_3 whose corresponding reception event at some node n is outside C_3. This message must therefore be a packet sent before point T_0 on node m. The marker for the channel from m to n is sent at or after point T_0 and is received by point T_3, so the packet transmission is reordered with respect to the marker, contradicting assumption (M2).

Let C_R be the set of events that strictly precede the T_0 event on every thread. We show that C_R is a consistent cut. If not, there is a send event outside C_R whose receive event lies inside C_R. As C_R has no update events, this must be a packet transmission; hence, its send event must be outside C_3. But that contradicts the consistency of C_3.

As $C_R \subset C_3$ and both are consistent, there is a linearization y of this computation where the state s corresponding to C_R occurs before the state t corresponding to C_3. By the properties of C_R and C_3, all events on y prior to s occur at Red nodes; all update events occur between s and t; state t is quiescent as it corresponds to cut C_3; and all nodes are at the Green configuration in t. Thus, y may be divided into the three segments described in the proof sketch. □

Corollary 2.2. *The update algorithm satisfies per-packet consistency.*

Proof. Consider any packet history in a run x with a completed update. This history consists of a sequence of transmissions that constitute a path through the computation for x. By Theorem 2.1, there is a reordering y of x where the update appears to be instantaneous and the dependencies between packet events are preserved. As the update takes effect in y at a quiescent state, the packet history cannot cross the update point in y. Hence, the entire history must lie either in the segment prior to the update (*i.e.,* within the Red configuration) or in the segment after the update (*i.e.,* within the Green configuration), ensuring per-packet consistency. □

2.4.3 Remark I: Update State. A small amount of state must be maintained by the update shell process at each node. Specifically, the shell must track whether the configured process is Red or Green; and once the configured process turns Green, it must remember the color (red or green) of each input channel. This state can be erased once the update is complete.

2.4.4 Remark II: Distributed Snapshots. Readers familiar with the distributed snapshot algorithm of Chandy and Lamport [8] and its proof by Dijkstra [12] using a commutativity argument will recognize their strong influence on the construction of the causal update algorithm and its proof. Our algorithm builds on the central discovery of the Chandy-Lamport work: that a global consistent cut can be computed in a local, distributed manner. The objectives of the algorithms are, however, quite different. The goal of the snapshot algorithm is to construct a consistent global state; to do so, the algorithm records all packets that appear on a channel before the marker. Our update algorithm instead drops all such packets. Our atomicity proof requires additional commutativity reasoning, using the ideas of left- and right-movers originated by Lipton [25]. Correctness is shown in the more general setting of lossy, non-FIFO channels; the snapshot algorithms assume perfect FIFO channels.

2.5 The Inevitability of Packet Drops

We now turn to show that packet drops are unavoidable for any in-place, on-the-fly, consistent update method. This impossibility result relies on the following precise statement of the "on-the-fly" property. An update algorithm is on-the-fly if it is always possible for a node that has not yet updated to process a packet if one is available for processing at one of its input channels. Informally, the update algorithm cannot completely block normal network processing at any non-updated node.

Say that an update algorithm is *perfect* if, for any network and any configuration update, the algorithm operates in place, on the fly, guarantees per-packet consistency, and does not forcibly drop packets.

Theorem 2.3. *There is no perfect network update algorithm.*

Proof. The proof considers a particular network and particular configuration update as a counterexample to perfect behavior. The network is the one from Figure 1, with two types of packets, called α and β (distinguished, for instance, by their eventual destination).

The network is configured as follows. In the Red configuration, the North (N) node requires packets of type α to exit the network, while β packets pass through; the South (N) node allows α packets to pass through, while requiring β packets to exit the network. In the Green configuration, the functionality is flipped: α packets now exit through the South node, while β packets exit through the North node. In both configurations, the East (E) and West (W) nodes pass every packet through unchanged.

Consider any update algorithm that operates on the fly and in place and does not drop any packets. We show that it must violate per-packet consistency. Consider a completed update execution. As the execution is asynchronous and interleaving, updates to the North and South nodes occur at distinct points on the execution.

Say that the North node is the first to be updated. At the state (say s) immediately after this update point, the South node is still in its Red configuration. Now consider the following sequence of transitions from state s. An α packet enters at the West node. It transits through the South node (which is Red, so it lets the α packet through). The packet then transits through the East node and reaches the North node. As the North node is Green, the α packet also passes through this node, arriving back at the West node. This constitutes a routing loop. As neither the Red nor the Green configurations have routing loops, this packet trace is neither entirely Red nor entirely Green; thus, it violates per-packet consistency. This sequence of transitions is enabled thanks to the non-blocking assumption and as no packets are dropped during the update.

If the South node is the first to update, a symmetric argument shows that a fresh β packet (entering from either the East or the West node) traces a routing loop. Thus, per-packet consistency is violated in either case. □

It is interesting to see how the causal update algorithm handles the first case. The α packet will make its way to the North node; however, it will then be dropped by the North node, as the node has turned Green but the channel from E to N is red. This prevents the routing loop from being formed.

2.6 Optimizations

Although packet drops are inevitable in general, there are cases where packet drops can be avoided or limited.

2.6.1 Acyclic Networks. An acyclic network is one where the network graph does not have a cycle. For an acyclic network, we make two modifications to the general algorithm: (1) Step 2 is modified

so that point T_1 (when a node turns Green) is chosen to be the point *after every input channel has received a marker*, and (2) The initial nodes are chosen to be precisely those nodes that do not have an incoming edge.

It is worth noting that this modification may lead to a deadlock in general graphs. For instance, consider the network in Figure 1 as a sub-network. Say that markers reach nodes E and W. The modified rule would require node W to wait for a marker from node N and for node E to wait for a marker from node S before turning Green and sending markers. But those markers must originate from nodes E and W, respectively, leading to a deadlock.

In an acyclic network there cannot be such a wait-for cycle, so the markers eventually propagate from the initial nodes to all other network nodes and edges. The modification to Step 2 ensures that all channels are green at Step 4; hence, no packets are dropped.

2.6.2 Network Structure. Updates typically do not change the functionality of every network node. Yet, the general algorithm may cause nodes whose functionality is unchanged to drop packets. In the extreme case, the trivial update that keeps all network functions unchanged may still result in packet drops. We show how a simple analysis of the structure of the network can reduce the number of nodes that are subject to packet drops and, in the case of the trivial update, eliminate all drops.

A natural 'fix' is to allow a node with a trivial update (i.e., no change) to process *all* packets. However, this 'liberal' policy may lead to inconsistencies downstream in the network, as can be shown by considering a network cycle where a node with a trivial update is sandwiched between nodes with non-trivial updates. (Details in Appendix B.)

To avoid inconsistencies, we proceed as follows. First, decompose the network graph into its maximal strongly connected components (SCCs). Then, recursively, classify an SCC as liberal if all its nodes have a trivial update and all other SCCs reachable from it are liberal. Any SCC not classified as liberal is classified as conservative. Nodes in a liberal SCC follow the liberal policy, while nodes in conservative SCCs follow the general algorithm. It is an easy consequence that for a fully trivial update, all nodes follow the liberal policy, so there are no forced packet drops.

Theorem 2.4. *Let x be an infinite fair execution with a completed structure-dependent update. There is an infinite fair execution y that is a reordering of events on x on which the update appears to take effect instantaneously. The instantaneous update state may not be quiescent, but any pending packet transmissions are subsequently processed only by nodes with unchanged functionality. Executions x and y agree on the ordering of non-update events.*

Proof. (Sketch) The proof proceeds as in the proof of Theorem 2.1. The consistent cut corresponding to the instantaneous update is as defined there. But now it is possible for this cut to be non-quiescent, implying that a packet sent prior to the cut may be received after the cut. However, this transmission is only possible if the receiving node follows the liberal policy. Hence any subsequent processing of the packet is through nodes with unchanged functionality.

Now consider any packet history. Either that history lies within the update cut, in which case it is a fully Red history; or it lies outside the cut, in which case it is fully Green; or (in the situation discussed above) it is partly within the update cut and partly outside the cut. Then the portion of the history that is outside the update cut passes only through nodes with no new functionality, so that the entire history can be considered to be Red. This establishes per-packet consistency. □

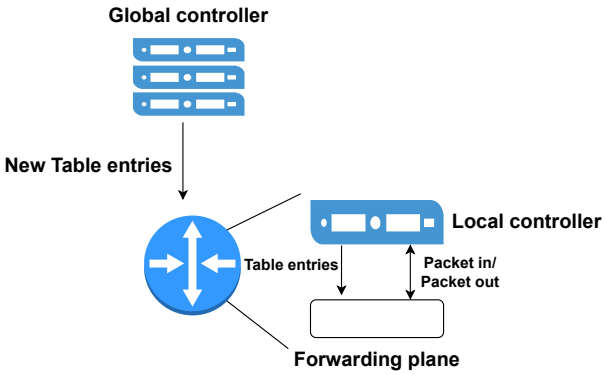

Figure 3: Switch and local controller relation.

Slight modifications to the causal update algorithm (described in Appendix C) result in updates that are resilient to link and node failures short of a network partition.

3 IMPLEMENTATION

The causal update algorithm requires state-keeping at each network node. The states help initiate the update, install new routing rules, process packets properly during the update, and determine when the update is complete. The special marker control packet is needed to control the update.

Using programmable software switches and a simple controller, we developed an SDN-based prototype that meets these requirements. Our implementation on the BMv2 Simple Switch requires only about 50 lines of P4 code [6, 10] on top of a programmable IP switch and around 100 lines of Python code on a simple controller. As shown in figure 3, the global controller offloads the table entries to the local controller, and the local controller can insert the new rules into the switch forwarding plane. The switch components communicate with each other via the packet in and out interface.

3.1 The Controller Component

An SDN-based network has a global controller with full visibility into the network. This global controller initiates the network update process by (1) selecting a set of switches that together can reach all other switches in the network and (2) sending a marker packet to each selected switch to start the update process on the switch.

Fetching the updated rules from the global controller adds latency. To hide this latency, the controller sends the updated rules to the local switch controller prior to the update: the update algorithm determines when the controller should install the new rules on the switch. Until that point, the new rules are stored on the local controller; this storage does not use the specialized, limited memory on the switch.

The global controller also tracks the progress of the network update. Each switch sends a notification to the global controller when its new ruleset is installed. Once all switches have been updated, the global controller marks the update as complete.

3.2 The Switch Component

The switches and the local controller are key components. To track the update, the P4 code in the switch maintains registers `Switch` and `Update` (both single bit), and `Port` (two bits per port).

A switch receives the marker packet from the global controller if it is one of the starting switches. Otherwise, the marker packet comes from another switch. Upon receiving the first marker packet, the switch sets the `Switch` register to indicate that the switch is experiencing an update, and triggers the causal update algorithm. While the `Switch` register is set, green packets will be buffered for processing later, and red packets will be forwarded by the current ruleset, implementing Step 3 of the algorithm.

For an input port, a single bit of the 2-bit `Port` register suffices to track whether a marker has been received on that port. For an output port, the `Port` register tracks three states to satisfy the M1 assumption of reliable marker transfer: *i.e.,* (1) a marker has not been sent on the output port, (2) a marker has been sent but no ack has been received, and (3) a marker has been sent and an ack has been received.

A single-bit register called `Update` records whether the new rules have been installed. This register marks Step 4 along with the `Switch` register. A network update is complete when this register is set in all network switches.

The marker packet is implemented with an Ethernet header and a marker header. The marker header contains 8 bits of update ID to differentiate between multiple updates.

An important aspect of the algorithm is to buffer incoming packets (Step 3a of the algorithm) We try to buffer the packets by forcing the switch to send back packets to itself, using the recirculation option of the programmable switch. By recirculating the packets, we can prevent some packet drops if the queue for each port reaches its capacity.

4 PERFORMANCE EVALUATION

We evaluate performance in the Mininet environment [22]. We consider *throughput* and *update completion time* as the key performance measures. By design, the causal update algorithm operates in-place and is thus memory-light.

4.1 Simulation Setup

We use two different topologies: a simplified Google's Wide Area Networks (WAN) topology with a high propagation delay and around 20 switches, and a Rocketfuel topology (AT&T, 7018) [33] with a moderate propagation delay and about 150 switches. The first is representative of a web-scaler topology; the second of a service-provider topology. We simplified Google's WAN [17] by aggregating switches that are close to each other; for instance, we have a single switch in Nebraska that represents five nearby switches. All links have 10 Gbps capacity. We generate the *Interactive* traffic from [16] between each pair of switches using TCP. As shown in Figure 4, we have two sets of routes, called old and new in the simplified Google's WAN. For the Google network, we chose two routes from Singapore to Finland, one of the longest pairwise routes. We estimate transmission delays and include them in the simulations for the Google WAN. We locate the Global Controller in Virginia and connect it to all the switches for Google's WAN. In AT&T Network, we update routes between Abingdon, VA, and Riverside, CA from the shortest path to the second shortest path. We put the global controller in Dallas, TX. The reported results are averages over 10 simulation runs; the variation across runs was at most 11%.

4.2 Throughput Evaluation

We test throughput in Google's WAN using a scenario in which we first send TCP traffic for 55 seconds. At $t = 20s$, we start updating the network as shown in Figure 4. We calculate the aggregated throughput at small time intervals. We use the throughput without any updates as the baseline. From the network structure, every switch is reachable from every other switch, so the choice of the initially updated nodes can be made in multiple ways. We consider three strategies and first present the results on the Google WAN.

In the first strategy, the global controller selects an initial switch at random. As Figure 5 shows, the throughput decreases shortly after the update starts, experiencing approximately a 33% reduction in the mean throughput (compared to no update). Random selection may choose a switch in the interior of the network, leading to more packet drops.

In the second strategy, the global controller starts the update from several switches at once. As shown in Figure 5, this decreases the mean throughput by 63%. We believe that the extra reduction in throughput is due to additional packet drops, as it is now more likely for red packets to be trapped between nodes with old and new configurations. The measured packet drop rate also increases by 44% compared to the random selection strategy.

The final strategy is for the global controller to start the update from switches at the network edge. This turns out to be the best strategy in terms of throughput loss; we believe this is so as the update "wavefront" follows the natural packet flow, resulting in more packets being processed with the updated ruleset. The mean throughput decreases by about 19% compared to the baseline.

The results on AT&T network show a similar trend: throughput losses for the three strategies are, correspondingly, 28%, 44%, and 15%. At the same time, despite the larger size of the AT&T network, its propagation delay is smaller, resulting in fewer packets being trapped. Thus, the throughput reduction for the AT&T topology is smaller than that for the Google topology. This suggests that the throughput loss may depend more upon the propagation delay than upon the size of the network.

4.3 Update Time Evaluation

The global controller tracks the switch updates to determine the overall update time. In theory, the update time is proportional to the network diameter, as marker packets must be distributed to every network node.

The baseline is the *one-shot* update, where nodes are updated in an arbitrary order. This has the least update time, although it does not guarantee consistency. Our experiments show that random

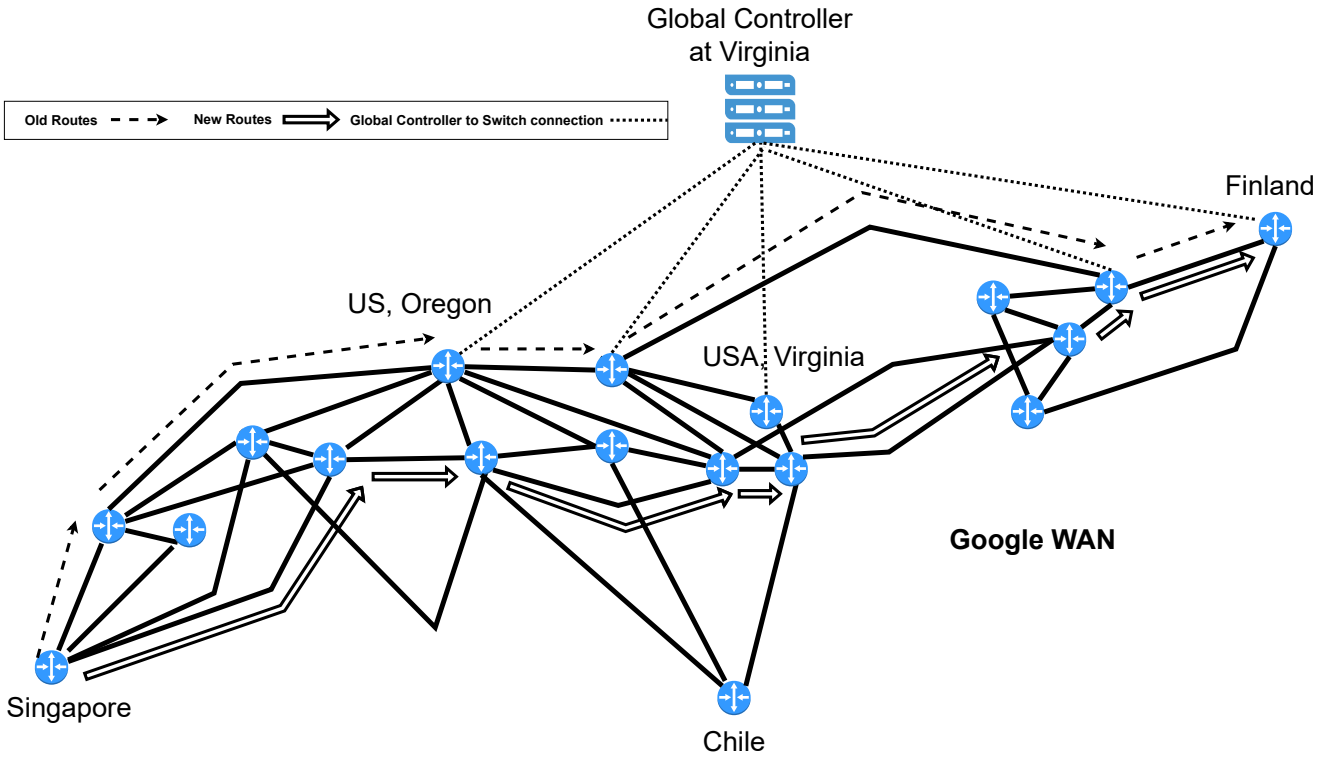

Figure 4: Simplified Google WAN network (from [17]) with route updates.

selection and edge selection impose the same latency, as the switch and network characteristics are the same. Figure 5 shows that on average the update time with edge-node selection has a 32% overhead, while multiple switch selection has a smaller 24% overhead for the Google WAN. (The overhead is 30% and 27%, respectively, for the AT&T network.)

The update time on the Google WAN is longer due to the added propagation delays. It also takes longer for the throughput to recover, as the propagation delay results in both the input and recirculation queue reaching capacity; we noted that on average about 50% of the recirculated packets were dropped, while in the AT&T network, only 30% of packet drops were due to queue overload.

These results point to a fundamental trade-off between the average update time and the mean throughput reduction in causal updates. In situations where the network requires a quick update, our results suggest following the multiple-switch approach. The edge-switch strategy is better for situations where limiting throughput loss has more importance.

5 RELATED WORK

There is a considerable literature on methods for network update, surveyed in depth in [14]. Focusing on correctness, one can broadly classify update methods into two groups: those that preserve a fixed property (such as no routing loops), and others that preserve a large class of properties. The two-phase update method of [31] and the causal update method are examples of the latter class: per-packet consistency ensures the absence of multiple errors, such as routing loops, black holes, and routing-policy violations.

We first discuss methods of the latter type that preserve a large class of properties. The pioneering work is that in [31], which formulates the important notion of per-packet consistency and provides the two-phase update method. As discussed, this method is consistent, on-the-fly, and does not drop packets, but it does not operate in place, essentially doubling the memory required at each switch. Follow-on work in [19] trades-off the time required to perform a consistent update against the rule-space overhead. This algorithm divides an update into a sequence of multiple (small) updates, where each update "slice" (their term) is installed using the two-phase solution. This reduces the memory overhead as memory doubling is only required per slice, but also slows down the update, and careful analysis (using Mixed Integer Programming) is needed to satisfy consistency through the slice-by-slice process. A different proposal in [30] sends every packet that would be handled differently by the two sets of rules to the SDN controller, which decides how it should be processed. This method is impractical, as it inserts the controller into the data plane, imposing substantial overhead on packet processing and seriously limiting throughput.

The *suffix causal consistency* (SCC) criterion [26] weakens per-packet consistency to require only that if the actual path taken by a packet passes through an updated rule, then it must share a *suffix* with the path that the packet would have taken through the updated network. That suffices to enforce black-hole freedom, bounded looping, and some forms of waypoint policies. The SCC

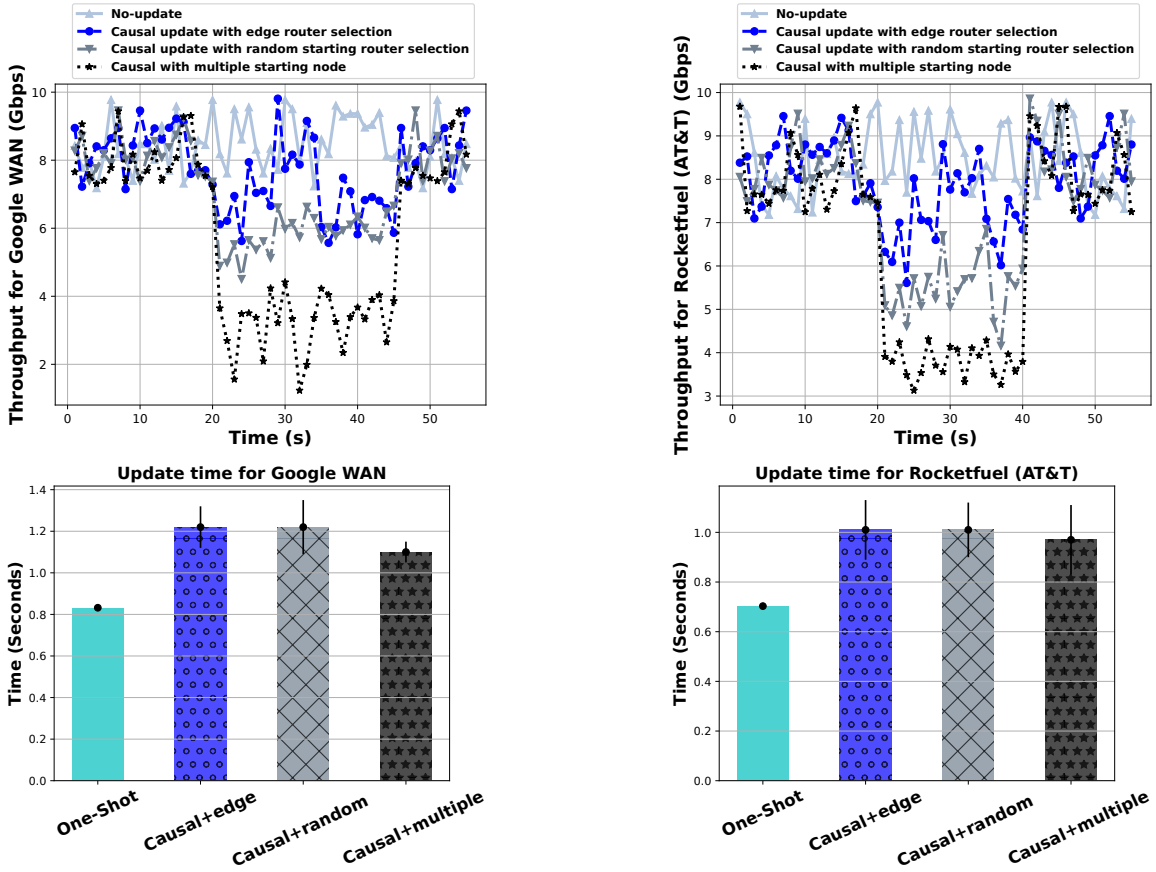

Figure 5: Causal Update Performance. Starting the network update from multiple switches results in a quicker update, while starting the update from the edge switches lowers throughput loss.

algorithm is based on a (worst-case quadratic-time) analysis of the current and updated rule sets. It also requires adding an integer timestamp to every packet.

The work in [28] considers trade-offs between the strength of consistency properties and the efficiency of enforcement. The causal update algorithm satisfies the strongest criterion of packet-coherence; hence, it also satisfies loop-freedom. It satisfies drop-freedom in the narrow sense in which it is defined (where a packet is dropped if there is no matching rule), but not in the broader sense that no packets are ever dropped during an update. Causal update also satisfies the memory-limit criterion as it operates in place. The bandwidth-limit criterion is difficult to analyze as that depends on assumptions about input traffic patterns and requires a network-wide analysis of congestion.

The other category of update methods are those that work in place, but preserve a fixed set of properties. Work in [15, 28] suggests an interesting approach for synthesizing in-place updates. In this approach, a dependency graph is built for the update, whose structure varies with the network and the strength of the desired consistency guarantee. This graph is analyzed to extract an update sequence. It is shown that the question is NP-hard in general, even when restricted to loop-freedom. On the other hand, a per-packet consistent update sequence can be found in polynomial time [7]. Other work in this vein may be found in [13, 27]. It should be noted that update sequences satisfying the desired consistency property need not exist in all networks; hence, these algorithms could fail to find an acceptable update sequence.

Other methods for synthesizing in-place updates include [18, 34, 37]. We focus on the Customizable Consistency Generator (CCG) method of [37], which is possibly the most flexible. This method is given a fixed set of properties and attempts to synthesize an in-place update plan that would safely meet all properties. The synthesis is carried out by reduction to a network verification question. Once a safe update plan has been found, CCG greedily processes network state transitions to heuristically minimize update time. However, the synthesizer may fail to find an in-place plan (and indeed by our impossibility result, there may not exist any such plan); in that case, the CCG method falls back to the two-phase process.

Several automatic update synthesis tools based on these and other algorithms have been developed for constructing in-place update sequences (e.g., NetSynth [29], FLIP [35], and AllSynth [23]). Work in [24] explores the additional dimension of security, aiming to protect a network from updates initiated by a malicious controller.

These in-place update methods based on synthesis may fail to find a suitable update sequence for some networks. The causal update algorithm works over all networks and does not require potentially expensive pre-processing or synthesis.

The network update question is connected to the broader question of consistently updating an active hardware or software system. This question has been investigated in several settings: the most relevant is that of component-wise update of an active distributed system (cf. [1, 20]). It is remarkable to observe certain recurring themes: distributed systems are updated using multi-version labeling [1], analogous to the two-phase network update; consistency is defined with respect to external observers, analogous to consistency with respect to observable packet histories; and operations on software objects may be blocked to ensure consistency [1], analogous to packet drops. Nonetheless, methods for updating distributed systems are more complex and do not easily transfer to the network setting. In the other direction, the causal update algorithm could be applied to update components of an active distributed system; but such a system must be carefully designed to tolerate occasional message loss. Networks have that property by design.

ACKNOWLEDGMENTS

We thank the anonymous reviewers and our shepherd, Ratul Mahajan, for their helpful comments. We would also like to thank T. V. Lakshman, Mahesh Marina, and Scott Smith for insightful comments and helpful suggestions on an earlier draft of this paper.

REFERENCES

[1] Sameer Ajmani, Barbara Liskov, and Liuba Shrira. 2006. Modular Software Upgrades for Distributed Systems, Dave Thomas (Ed.). Springer.
[2] Mohammad Al-Fares, Alexander Loukissas, and Amin Vahdat. 2008. A Scalable, Commodity Data Center Network Architecture *(SIGCOMM)*.
[3] Mohammad Al-Fares, Sivasankar Radhakrishnan, Barath Raghavan, Nelson Huang, and Amin Vahdat. 2010. Hedera: dynamic flow scheduling for data center networks *(NSDI'10)*. USENIX Association.
[4] Özalp Babaoğlu and Keith Marzullo. 1993. *Consistent Global States of Distributed Systems: Fundamental Concepts and Mechanisms.* ACM Press/Addison-Wesley Publishing Co.
[5] Theophilus Benson, Ashok Anand, Aditya Akella, and Ming Zhang. 2011. MicroTE: fine grained traffic engineering for data centers *(CoNEXT '11)*.
[6] Pat Bosshart, Dan Daly, Glen Gibb, Martin Izzard, Nick McKeown, Jennifer Rexford, Cole Schlesinger, Dan Talayco, Amin Vahdat, George Varghese, and David Walker. 2014. P4: Programming Protocol-Independent Packet Processors. *SIGCOMM Comput. Commun. Rev.* (2014).
[7] Pavol Cerný, Nate Foster, Nilesh Jagnik, and Jedidiah McClurg. 2016. Optimal Consistent Network Updates in Polynomial Time *(Lecture Notes in Computer Science)*. Springer.
[8] K. Mani Chandy and Leslie Lamport. 1985. Distributed Snapshots: Determining Global States of Distributed Systems. *ACM Trans. Comput. Syst.* (1985).
[9] Shawn Shuoshuo Chen, Keqiang He, Rui Wang, Srinivasan Seshan, and Peter Steenkiste. 2024. Precise Data Center Traffic Engineering with Constrained Hardware Resources. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*.
[10] P4 Language Consortium. 2023. P416 Language Specification. (2023). https://p4.org/p4-spec/docs/P4-16-v1.0.0-spec.html Accessed: 2023-11-30.
[11] Andrew R. Curtis, Jeffrey C. Mogul, Jean Tourrilhes, Praveen Yalagandula, Puneet Sharma, and Sujata Banerjee. 2011. DevoFlow: scaling flow management for high-performance networks. *SIGCOMM Comput. Commun. Rev.* (2011).
[12] Edsger W. Dijkstra. 1983. *The distributed snapshot of Chandy/Lamport/Misra.* Technical Report. University of Texas at Austin.
[13] Szymon Dudycz, Arne Ludwig, and Stefan Schmid. 2016. Can't Touch This: Consistent Network Updates for Multiple Policies. In *DSN*. IEEE Computer Society.
[14] Klaus-Tycho Foerster, Stefan Schmid, and Stefano Vissicchio. 2019. Survey of Consistent Software-Defined Network Updates. *IEEE Communications Surveys & Tutorials* (2019).
[15] Klaus-Tycho Förster, Ratul Mahajan, and Roger Wattenhofer. 2016. Consistent updates in software defined networks: On dependencies, loop freedom, and blackholes. In *Networking*. IEEE Computer Society.
[16] Chi-Yao Hong, Srikanth Kandula, Ratul Mahajan, Ming Zhang, Vijay Gill, Mohan Nanduri, and Roger Wattenhofer. 2013. Achieving High Utilization with Software-Driven WAN. *SIGCOMM Comput. Commun. Rev.* (2013).
[17] Chi-Yao Hong, Subhasree Mandal, Mohammad Al-Fares, Min Zhu, Richard Alimi, Kondapa Naidu B., Chandan Bhagat, Sourabh Jain, Jay Kaimal, Shiyu Liang, Kirill Mendelev, Steve Padgett, Faro Rabe, Saikat Ray, Malveeka Tewari, Matt Tierney, Monika Zahn, Jonathan Zolla, Joon Ong, and Amin Vahdat. 2018. B4 and after: Managing Hierarchy, Partitioning, and Asymmetry for Availability and Scale in Google's Software-Defined WAN. In *Proceedings of the 2018 Conference of the ACM Special Interest Group on Data Communication (SIGCOMM '18)*. Association for Computing Machinery.
[18] Xin Jin, Hongqiang Harry Liu, Rohan Gandhi, Srikanth Kandula, Ratul Mahajan, Ming Zhang, Jennifer Rexford, and Roger Wattenhofer. 2014. Dynamic Scheduling of Network Updates. In *Proceedings of the 2014 ACM Conference on SIGCOMM (SIGCOMM '14)*. Association for Computing Machinery.
[19] Naga Praveen Katta, Jennifer Rexford, and David Walker. 2013. Incremental Consistent Updates. In *Proceedings of the Second ACM SIGCOMM Workshop on Hot Topics in Software Defined Networking (HotSDN '13)*. Association for Computing Machinery.
[20] Jeff Kramer and Jeff Magee. 1990. The Evolving Philosophers Problem: Dynamic Change Management. *IEEE Trans. Software Eng.* 11 (1990).
[21] Umesh Krishnaswamy, Rachee Singh, Paul Mattes, Paul-Andre C. Bissonnette, Nikolaj S. Bjørner, Zahira Nasrin, Sonal Kothari, Prabhakar Reddy, John Abeln, Srikanth Kandula, Himanshu Raj, Luis Irún-Briz, Jamie Gaudette, and Erica Lan. 2023. OneWAN is better than two: Unifying a split WAN architecture. In *NSDI*. USENIX Association.
[22] Bob Lantz, Brandon Heller, and Nick McKeown. 2010. A Network in a Laptop: Rapid Prototyping for Software-Defined Networks. In *Proceedings of the 9th ACM SIGCOMM Workshop on Hot Topics in Networks (Hotnets-IX)*. Association for Computing Machinery.
[23] Kim G Larsen, Anders Mariegaard, Stefan Schmid, and Jiří Srba. 2023. AllSynth: A BDD-based approach for network update synthesis. *Science of Computer Programming* (2023).
[24] James Lembke, Srivatsan Ravi, Pierre-Louis Roman, and Patrick Eugster. 2023. Secure and Reliable Network Updates. *ACM Trans. Priv. Secur.* 26, 1 (2023).
[25] Richard J. Lipton. 1975. Reduction: A Method of Proving Properties of Parallel Programs. *Commun. ACM* 18, 12 (1975).
[26] Sheng Liu, Theophilus A. Benson, and Michael K. Reiter. 2019. Efficient and Safe Network Updates with Suffix Causal Consistency. In *EuroSys*. ACM.
[27] Arne Ludwig, Jan Marcinkowski, and Stefan Schmid. 2015. Scheduling Loop-free Network Updates: It's Good to Relax!. In *PODC*. ACM.
[28] Ratul Mahajan and Roger Wattenhofer. 2013. On consistent updates in software defined networks. In *HotNets*. ACM.
[29] Jedidiah McClurg, Hossein Hojjat, Pavol Cerný, and Nate Foster. 2015. Efficient synthesis of network updates. In *PLDI*. ACM.
[30] Rick McGeer. 2012. A Safe, Efficient Update Protocol for Openflow Networks. In *Proceedings of the First Workshop on Hot Topics in Software Defined Networks (HotSDN '12)*. Association for Computing Machinery.
[31] Mark Reitblatt, Nate Foster, Jennifer Rexford, Cole Schlesinger, and David Walker. 2012. Abstractions for Network Update *(SIGCOMM '12)*. Association for Computing Machinery.
[32] Arjun Roy, Hongyi Zeng, Jasmeet Bagga, George Porter, and Alex C Snoeren. 2015. Inside the Social Network's (Datacenter) Network. In *SIGCOMM*.
[33] Neil Spring, Ratul Mahajan, and David Wetherall. 2002. Measuring ISP topologies with Rocketfuel *(SIGCOMM '02)*. Association for Computing Machinery.
[34] Radhika Sukapuram and Gautam Barua. 2016. PPCU: Proportional per-packet consistent updates for Software Defined Networks. In *2016 IEEE 24th International Conference on Network Protocols (ICNP)*.
[35] Stefano Vissicchio and Luca Cittadini. 2016. FLIP the (Flow) table: Fast lightweight policy-preserving SDN updates. In *INFOCOM*. IEEE.
[36] Junlan Zhou, Malveeka Tewari, Min Zhu, Abdul Kabbani, Leon Poutievski, Arjun Singh, and Amin Vahdat. 2014. WCMP: weighted cost multipathing for improved fairness in data centers *(EuroSys '14)*. Association for Computing Machinery.
[37] Wenxuan Zhou, Dong Jin, Jason Croft, Matthew Caesar, and P. Brighten Godfrey. 2015. Enforcing Customizable Consistency Properties in Software-Defined Networks. In *12th USENIX Symposium on Networked Systems Design and Implementation (NSDI 15)*. USENIX Association.

Appendices are supporting material that has not been peer-reviewed.

A REORDERED COMPUTATION

Figure 6 shows the result of reordering events according to the proof on the example update computation shown in Figure 2. The reordering moves the reception of the marker m_1 to the right over the transmission of q on the timeline for S, and moves the reception of marker m_0 to the left over the reception and transmission of packet s on the timeline for W.

B HANDLING TRIVIAL UPDATES

Consider a directed anticlockwise cycle of three nodes (say $A \rightarrow B \rightarrow C$) where C is the only node with a trivial update. Let X_c represent the functionality of node x with color $c \in \{r, g\}$. Now consider the following scenario: a packet p is injected at B before the update; that produces a packet $B_r(p)$ on the channel to C; node C is updated; packet $C_g(B_r(p))$ is generated on the channel to A; node A updates; packet $A_g(C_g(B_r(p)))$ is generated on the channel to B; node B updates. For per-packet consistency, it should be possible to express the transformation of packet p as either an all-green transformation, *i.e.,* as $A_g(C_g(B_g(p)))$, or as an all-red transformation, *i.e.,* as $A_r(C_r(B_r(p)))$. Although it is the case that $C_g = C_r$, it is possible to choose the green functionality for nodes A and B such that neither case holds, leading to a failure of per-packet consistency.

C RESILIENCE TO NETWORK CHANGES

We describe slight modifications of the causal update algorithm that make it resilient to network changes that may occur during the update process.

Link or Node Failures. The model on which these algorithms are based implicitly handles the case of an intermittently faulty connection. Condition (M1) says that marker packets must be transmitted reliably. In practice, this may be ensured by requiring an acknowledgment for each marker transmission across a link and retrying the transmission until the acknowledgment is received. Thus, if a link fails before or during a marker transmission, the transmission is retried until an acknowledgment is received. A permanent link disconnection may partition the network. That could lead to a situation where the one half of the partition has updated to the Green configuration while the other half remains in its Red configuration as it has not received a marker. This situation must be recognized and handled outside the algorithm (e.g., by the network management software) and the update process must be restarted in the Red portion of the network. A node failure disconnects all links to and from that node. We may view this as a sequence of link disconnections and handle it accordingly.

Node and Link Additions. On the addition of a new link, if its source node has updated to its Green configuration, a fresh marker is sent along that link. This ensures that all newly connected nodes are reached by the update. A new node is simply initialized to its Green configuration.

D HANDLING MULTIPLE UPDATES

The causal update algorithm handles a single update, from the Red to the Green configuration. A subsequent update starts from the Green configuration, which now plays the role of the old (Red) configuration in the causal update algorithm.

Although unlikely to arise in practice, multiple concurrent updates can be handled by linearly ordering the updates. A marker now carries the update "level," as does each node. On receiving a marker with update level k, say, a node that is at version level m proceeds as follows (generalizing the handling of red and green channels): if $k > m$, the node moves to update version k; marks the channel on which it has received the marker as a level-k channel; sends out markers with level k; and only processes packets from input channels at level k, dropping packets from input channels at lower levels. If $k = m$, the node marks the channel as a level-k channel and starts processing packets from it. If $k < m$, the node drops the marker and remains at update m. As a result, a packet history passes only through nodes of the same update level. Interestingly, a node may skip an update, e.g., updating directly from level 1 to level 3. Ultimately, all nodes will update to the maximum level.

E DATACENTER TOPOLOGY EVALUATION

We evaluated the causal update algorithm in an 8-ary fat-tree [2] topology with 80 switches, as a representative datacenter topology. The propagation delay in datacenter topologies is negligible, making them different from WANs. The workload pattern is a cache follower traffic from [32] consisting of large and small traffic: 50% of the traffic is smaller than 50KB, with the rest ranging between 50KB and 10000KB. We consider the flows larger than 50KB as elephant flows, and we perform WCMP (weighted cost multi path) forwarding [36] that is prevalent in data center networks for elephant-flow [3, 5, 11]. With WCMP, traffic can be controlled in a finer granularity with topology changes and can be balanced between paths [9, 36]. We randomly choose 16 Top of the Rack (ToR) switches and update the WCMP weights at $t = 20s$. In mice flows, the default ECMP entries are used, so they are not a part of the update procedure. The results are an aggregate over 10 repetitions and the variation across them is less than 5%.

The evaluation results on the datacenter topology, shown in Fig. 7, are similar to the results on WAN simulations in terms of throughput and total update time. The throughput loss is the least (14%) for the edge router selection, while the total update time is the least when the update is started from multiple points. However, as the propagation delay is not significant, the overall throughput loss is smaller than for the AT&T topology, which has nearly twice the number of switches. For the same reason, the update time is significantly smaller than for the WAN scenarios. These results support the hypothesis that propagation delay affects performance more than the network scale.

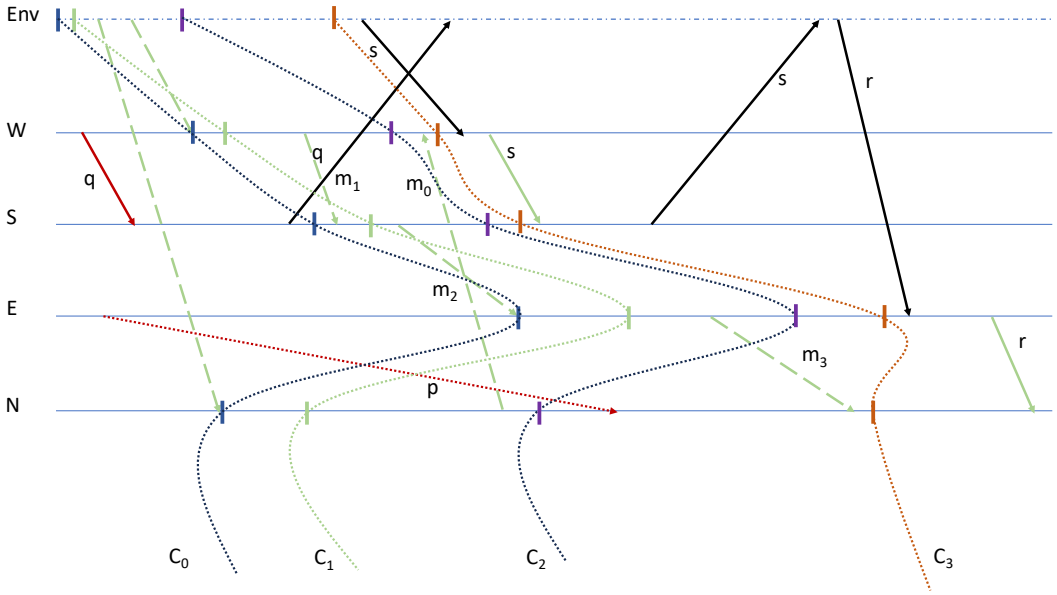

Figure 6: The space-time diagram for the reordered computation from Figure 2.

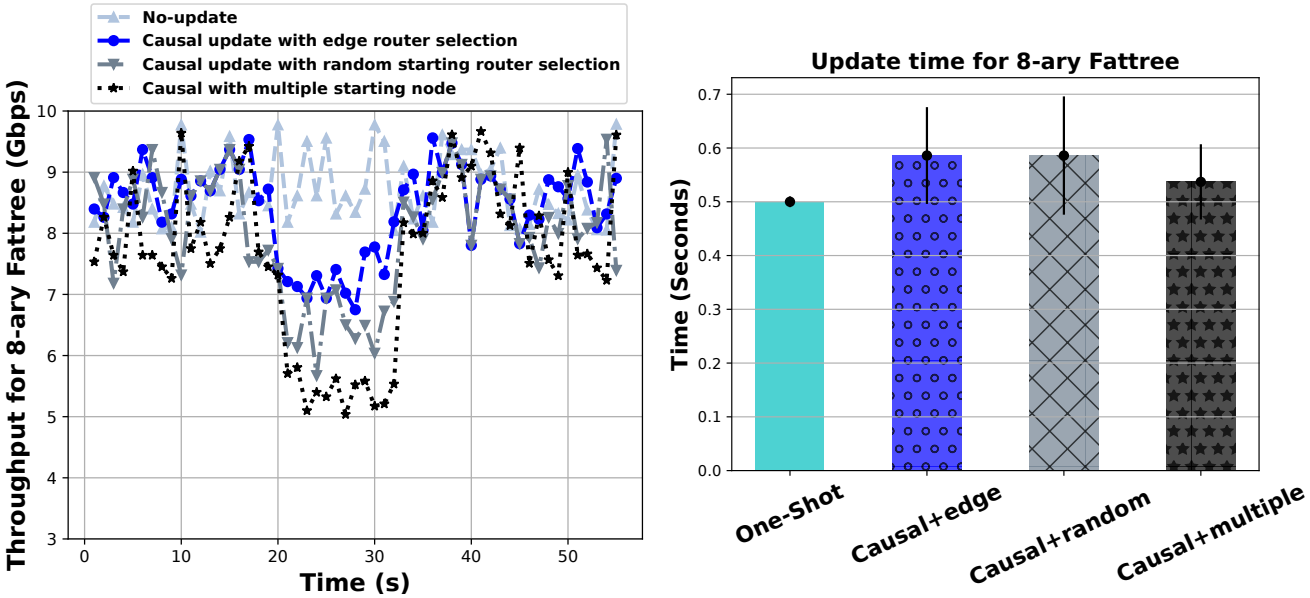

Figure 7: Causal Update Performance in fat-tree topology. The throughput reduction is the least when we start the update from edge routers, and the total update time is the minimum with the multiple router selection approach.